Binary Market Data File Specification

Stream Structure, Memory Mapping & Technical Reference

File Type: Binary Stream Capture (.bin)
Byte Order: Little-Endian
Packing: #pragma pack(push, 1)
Depth Level: FEED_LEVEL_DEPTH = 5
Target Struct: CONTRACT_DETAILS
Status: Validated Specification

1. Executive Overview

This document provides the low-level technical specification for reading and parsing the binary market data capture file (`.bin`). The file records sequential Level-2 market depth snapshots alongside contract master specifications. Every record in the file is framed with a size header followed by a timestamped record payload.

Key Integration Notes

All multi-byte numeric values (e.g., `int32_t`, `uint64_t`) use native **Little-Endian** format. Variable member layouts utilize standard C/C++ primitives. Strict byte packing rules apply to internal sub-structures.

2. Stream Structure & Framing Protocol

The binary file consists of a continuous stream of variable-sized blocks. Reading requires a two-step parsing loop: reading the framing length header, followed by the exact payload length specified.

BINARY RECORD FRAME LAYOUT

Header: nRead 8 Bytes (int64_t)	Payload Block (Length = nRead) [Receive Time (8B)] + [CONTRACT_DETAILS Data Structure]

Field Name	Data Type	Size (Bytes)	Description
`nRead`	`int64_t` / `ssize_t`	8	Specifies the total length of the payload immediately following this field. Signal EOF if read fails.
`RcTime`	`uint64_t`	8	Capture timestamp (Epoch nanoseconds/microseconds) located at offset 0 of the payload.
`CONTRACT_DETAILS`	`struct`	Variable (~282B)	Market depth order book and embedded contract details structure mapped directly from offset 8.

3. Payload Memory Layout: CONTRACT_DETAILS

The main structure contains top-5 bid/ask order book depth arrays, last trade details, bitfield status indicators, and embedded contract metadata.

Alignment & Padding Notice

> The root structure `CONTRACT_DETAILS` is **unpacked** by default in the compiler. However, the nested member `GenericContract` is wrapped inside a packed directive (`#pragma pack(push, 1)`). On 32-bit/64-bit architectures, 2 to 3 bytes of alignment padding may exist prior to the `Contract` offset.

Field Name	Data Type	Size (Bytes)	Description & Value Constraints
`BuyPrice[5]`	`int32_t[5]`	20	5 levels of bid prices (scaled by PriceExponent).
`BuyQty[5]`	`uint32_t[5]`	20	5 levels of bid quantities.
`SellPrice[5]`	`int32_t[5]`	20	5 levels of ask prices (scaled by PriceExponent).
`SellQty[5]`	`uint32_t[5]`	20	5 levels of ask quantities.
`NoOfBuyOrds[5]`	`int32_t[5]`	20	Number of buy orders at each depth level.
`NoOfSellOrds[5]`	`int32_t[5]`	20	Number of sell orders at each depth level.
`LastTradedPrice`	`int32_t`	4	Price of the latest executed transaction.
`Bitfield Container`	Bitfields	2	Contains `LastTradedQty:12` and `FeedEventAggressor:2`.
`Contract`	`GenericContract`	158	Packed contract specification structure (Section 4).

4. Nested Layout: GenericContract Struct (#pragma pack(1))

The `GenericContract` layout is strictly byte-packed without internal alignment gaps, yielding a fixed total size of **158 Bytes**.

Offset	Field Name	Data Type	Bytes	Description
0x00	`Exchange`	`Exchange_Type`	1	Enum (0 = EXCHG_CME, 1 = EXCHG_NONE)
0x01	`Index`	`int32_t`	4	System index identifier
0x05	`Token`	`int32_t`	4	Unique instrument token identifier
0x09	`ExpiryDate`	`int32_t`	4	Expiration date integer representation (YYYYMMDD)
0x0D	`StrikePrice`	`int32_t`	4	Strike price for option instruments
0x11	`LotSize`	`int32_t`	4	Contract lot size multiplier
0x15	`TickSize`	`int32_t`	4	Minimum price increment
0x19	`LowPriceRange`	`int32_t`	4	Lower circuit price limit
0x1D	`HighPriceRange`	`int32_t`	4	Upper circuit price limit
0x21	`BasePrice`	`int32_t`	4	Daily reference/closing price
0x25	`Multiplier`	`int32_t`	4	Contract value multiplier
0x29	`Margin`	`ContractMargin`	20	Nested margin structure (5 x `uint32_t`)
0x3D	`Symbol`	`char[50]`	50	Null-terminated ticker symbol string
0x6F	`SymbolCode`	`char[50]`	50	Null-terminated system symbol code string

Offset	Field Name	Data Type	Bytes	Description
0xA1	`InstrumentType`	`Instrument_Type`	1	Enum defining instrument class (See Section 5)
0xA2	`OptionType`	`Option_Type`	1	Option style identifier (CA, PA, CE, PE, NONE)
0xA3	`PriceExponent`	`uint8_t`	1	Price scaling exponent factor ($Price imes 10^{-Exponent}$)
0xA4	`PriceExponentDisplay`	`uint8_t`	1	Display decimal places formatting suggestion

5. Enumeration Reference Tables

5.1. Instrument_Type (`int8_t`)

Val	Enum Identifier	Val	Enum Identifier	Val	Enum Identifier
0	EQUITY	11	FUTIRT	22	OPTIVX
1	INDEX	12	FUTENR	23	OPTIRC
2	FUTIDX	13	FUTBLN	24	OPTIRD
3	FUTSTK	14	FUTBAS	25	OPTENR
4	FUTCOM	15	COM	26	OPTBLN
5	FUTCUR	16	OPTIDX	27	OPTBAS
6	FUTINT	17	OPTSTK	28	UNDINT
7	FUTIVX	18	OPTFUT	29	UNDCUR
8	FUTIRC	19	OPTCOM	30	UNDIRC
9	FUTIRD	20	OPTCUR	31	UNDIRT
10	FUTIRD	21	OPTINT	32	INST_ERROR

5.2. Option_Type (`int8_t`) & Exchange_Type (`int8_t`)

Value	Option_Type Label	Meaning
0	CA	Call American
1	PA	Put American
2	CE	Call European
3	PE	Put European
4	OPTION_NONE	Non-Option Instrument

Value	Exchange_Type Label	Meaning
0	EXCHG_CME	CME Group Exchange
1	EXCHG_NONE	Unassigned Exchange